LOCUS 2027
23rd National Technological Festival

# Day 2: Git & GitHub

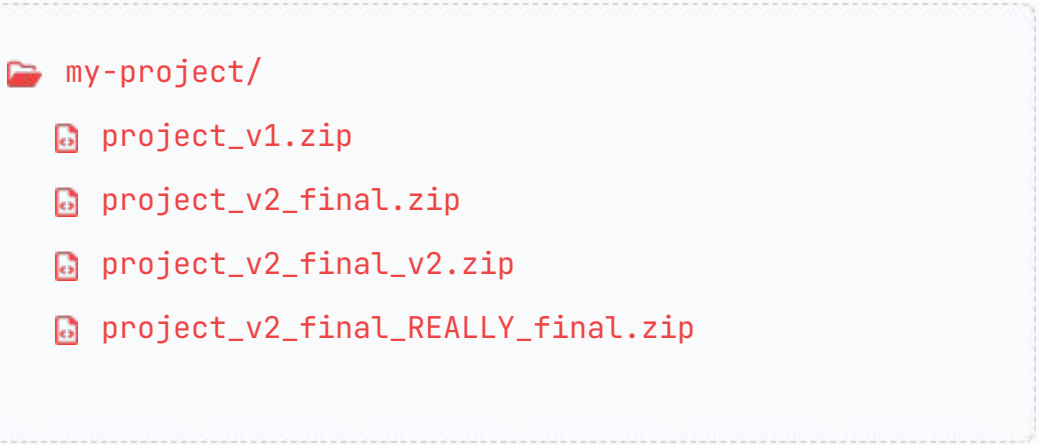
An introduction to modern version control, collaborative development, and core terminal logic.

GitHub

Data Hub

# LIFE BEFORE VERSION CONTROL






**To go back to the time** when your code was **working,** you have to **manually** revert everything

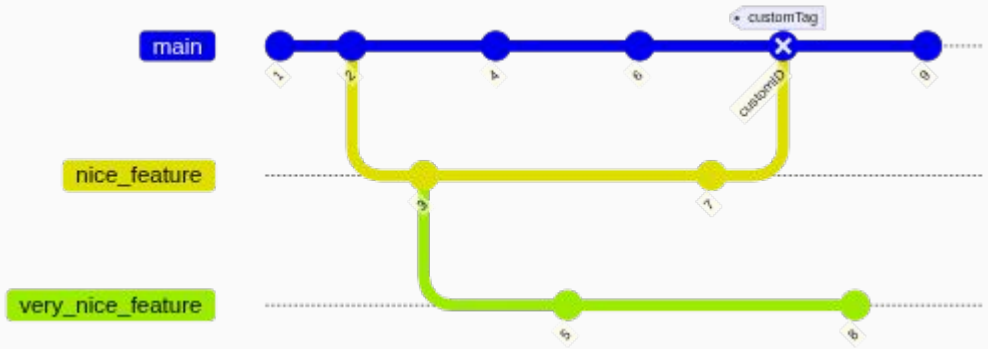
**Claude code** just broke your system, ruined your code, and **deleted a file**. How do you **fix everything** in three seconds?

# git restore .

Permanently discards all **uncommitted** modifications in your current working directory

# ENTER GIT VERSION CONTROL

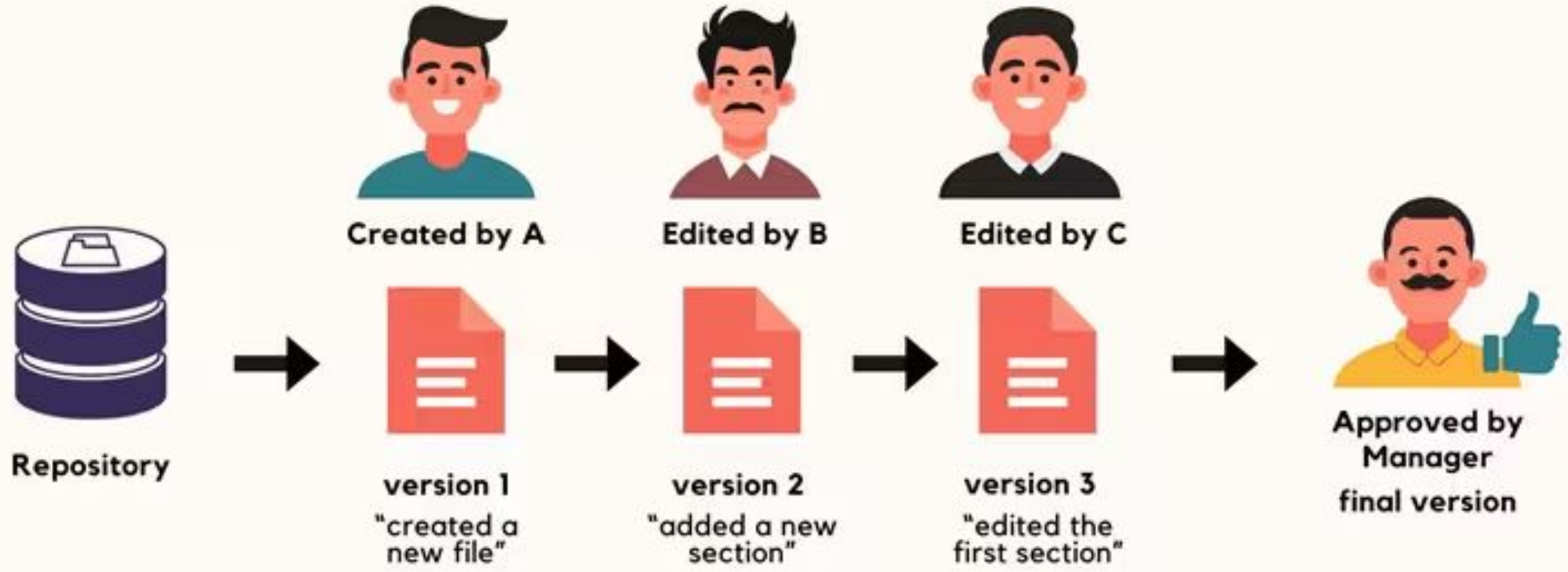
A structured database saving cryptographic **snapshots** of every code change across time.




```
Commit: 4a2b9f - Init repository skeleton
Commit: b3e5d1 - Build login feature layout
Commit: f8c2d7 - Fix CSS alignment bug
Commit: 2e4a81 - Setup secure validation
```

LOCUS 2027
23rd National Technological Festival

# VERSION CONTROL




GitHub

Data Hub

# GIT VS. GITHUB

LOCUS 2027
23rd National Technological Festival

| Feature | Git | GitHub |
| --- | --- | --- |
| **Purpose** | Local tracking software tool. | Cloud host platform for repositories. |
| **Interface** | Primarily CLI (Terminal). | Rich modern UI Web Dashboard. |
| **Collaboration** | Manual patch emails or file syncing. | Interactive Pull Requests & Issues. |
| **Automation** | Local hook script configurations. | Powerful cloud CI/CD pipelines (Actions). |

# CONFIGURING GIT

```
# Register your developer details globally
$ git config --global user.name "Alex Dev"
$ git config --global user.email "alex@dev.com"


# List current system configurations
$ git config --list
```

# GIT INIT



```
# Navigate to your workspace directory
$ cd dev-projects/site-v2


# Spin up a local Git ledger
$ git init
Initialized empty Git repository in /site-v2/.git/
```

# THE GIT LIFECYCLE

LOCUS 2027
23rd National Technological Festival


GitHub

Data Hub

# GIT ADD



```
# Stage-specific files for saving
$ git add main.js


# Stage all project adjustments instantly
$ git add .
```

# Remove from the staging area

```
# Unstage-specific files
$ git restore --staged <file-name>


# Unstage all staged changes
$ git restore --staged .

# Unstages the file and untracks it.
$ git rm --cached <file>
```

# GIT COMMIT

LOCUS 2027
23rd National Technological Festival

```
# Commit staged work with descriptive text
$ git commit -m "feat: login modal"
[main d8e2f1a] feat: login modal
 2 files changed, 24 insertions(+)
```

## Commit Snapshots

Takes a permanent cryptographic snapshot of staged changes. Your repository history is permanently extended with a unique, securely signed SHA-1 hash.

**Always write clear, present-tense, informative commit messages explaining what was done and why.**

GitHub

Data Hub

# GIT CLONE



```
# Copy a project down from a remote cloud host
$ git clone https://github.com/user/repo.git
Cloning into 'repo'...
 Receiving objects: 100% done.
 Resolving deltas: 100% done.
```

# GIT PUSH & GIT PULL



```
# Push local changes to remote main branch
$ git push origin main


# Pull remote cloud updates into local drive
$ git pull origin main
```

# BRANCHING & MERGING



```
# Create and jump to a feature branch
$ git checkout -b branch-login


# Merge branch work back into stable master
$ git checkout main
$ git merge branch-login
```

# THE DEVELOPER HANDSHAKE: README.MD

LOCUS 2027
23rd National Technological Festival

## What is a README?

```
# Minimal README.md
## Description
What & why...
## Installation
$ git clone https://github.com/...
```

> *"A good writer possesses not only his own spirit but also the spirit of his friends."*
>
> — FRIEDRICH NIETZSCHE

## Why it Matters

- **First Impression:** Your project's front face.
- **Focus Tool:** Defines the delivery scope.
- **Establishes Trust:** Removes execution friction.

GitHub

Data Hub

# ANATOMY OF A PROFESSIONAL README

LOCUS 2027
23rd National Technological Festival

01

## Identity

**Title:** Defines goals in one line.

**Description:** What it does and why.

**Features:** Core functionalities list.

02

## Operations

**Setup:** Local environment run steps.

**Usage:** Instructions and examples.

**Visuals:** App design mockups.

03

## Metadata

**Navigation:** Optional table of contents.

**Credits:** Collaborators and tutorials.

**License:** Code usage rights (e.g., MIT).

GitHub

Data Hub

We only learned how to save history.
But what if we encounter problems ?
How to go back in time ?

```
$ git log
$ git log -5
$ git log --author="Name"
$ git log --grep="bug fix"
```

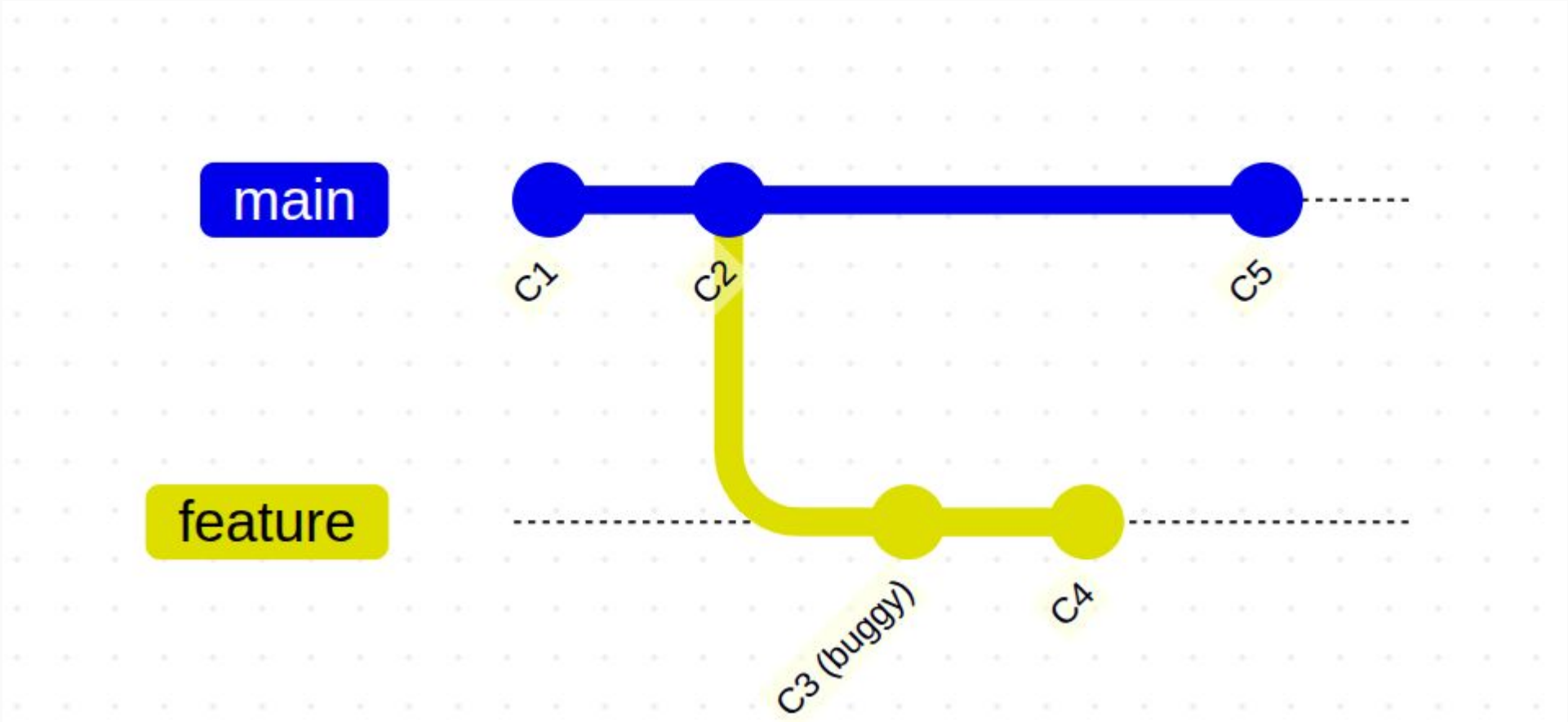
## Basic Rescue Moves (Local & Safe)

- `git restore <file>` – Discards unsaved changes in a specific file.
- `git checkout .` – Discards all unsaved changes in the entire project.
- `git commit --amend` – Edits the message or files of your very last commit.
- `git stash` – Saves and hides unfinished, messy work to clean your workspace.
- `git stash pop` – Brings back your hidden work exactly where you left off.

## Advanced Rescue Moves (History & Rewriting)

- `git revert <commit>` – Undoes an old commit by creating a safe, opposite commit.
- `git reset --soft <commit>` – Undoes commits but keeps your code changes staged.
- `git reset --hard <commit>` – Permanently deletes commits and wipes out all code changes.
- `git cherry-pick <commit>` – Copies a specific commit from another branch into yours.
- `git rebase <branch>` – Moves your entire branch history on top of another branch.

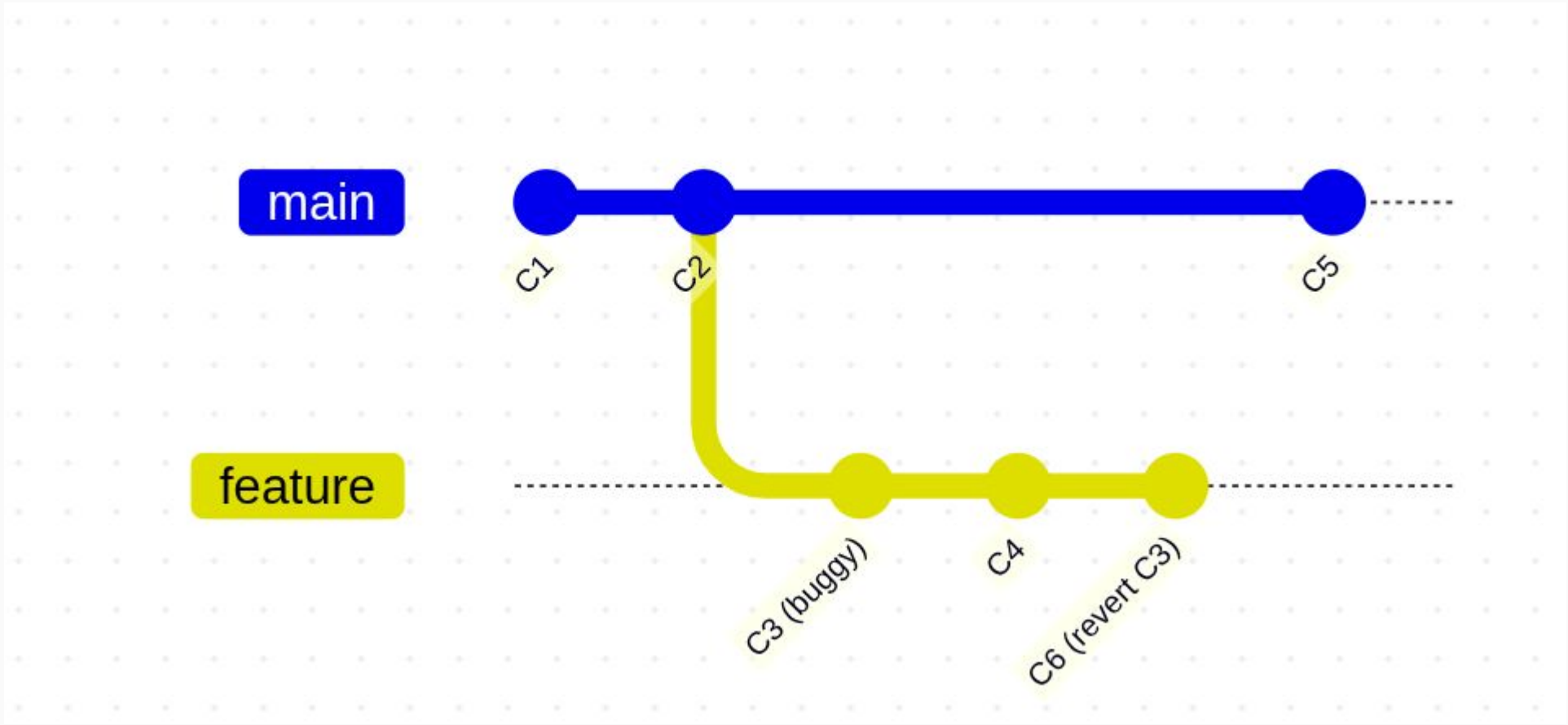
You pushed a feature to the production server yesterday, but it's causing bugs and you need to undo it immediately. You cannot delete the commit from the history because others have already pulled the code.


main
feature
C1
C2
C5
C3 (buggy)
C4

# git revert <commit>

# git revert C3





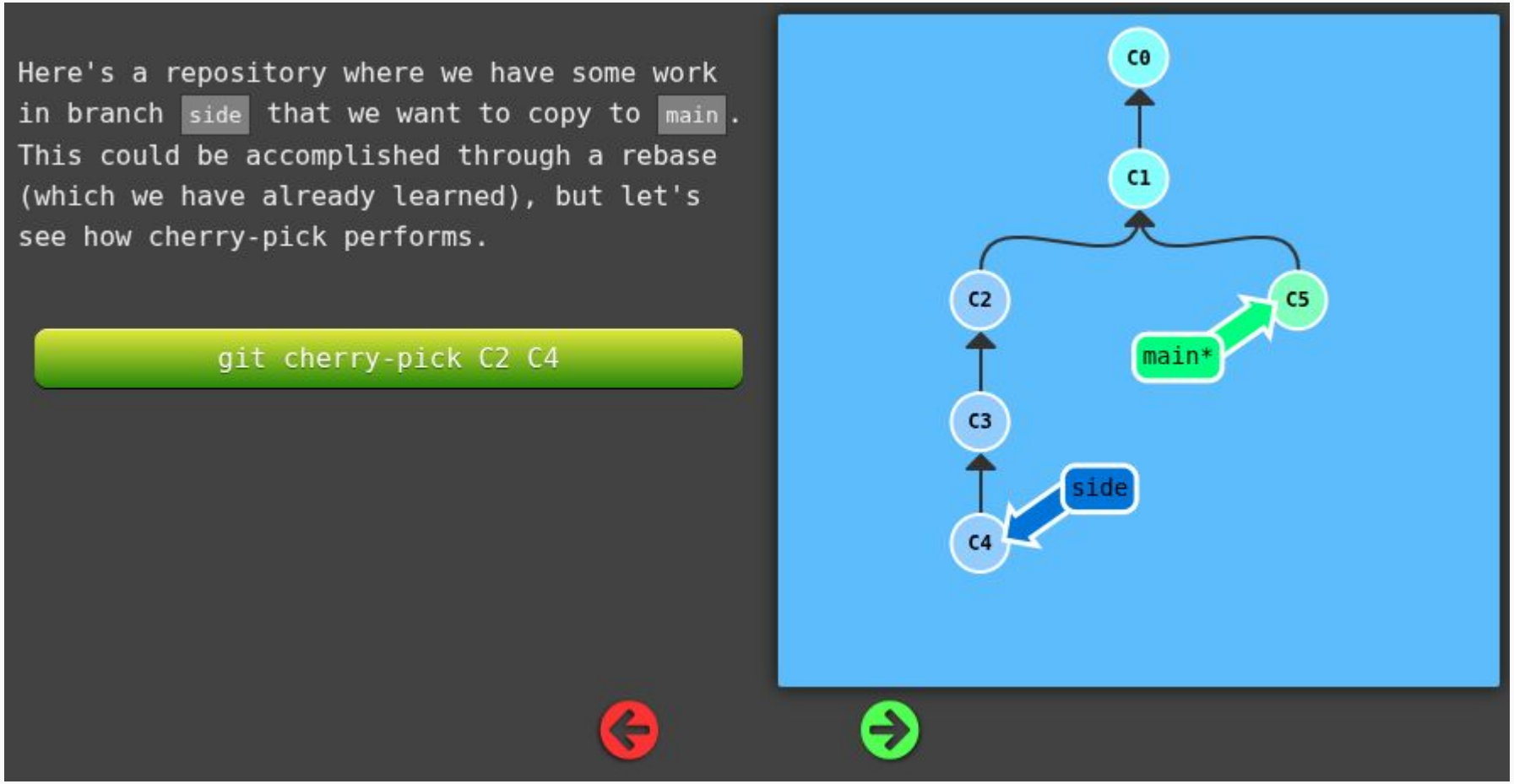
You just made three small commits in a row, but you realize they should have been combined into one clean commit. You want to undo those three commits and commit it again as one.

# git reset --soft <commit>

You started writing some experimental code, made a few local commits, and completely messed up your project. You want to throw away everything you've done since yesterday morning and start fresh from that clean point.

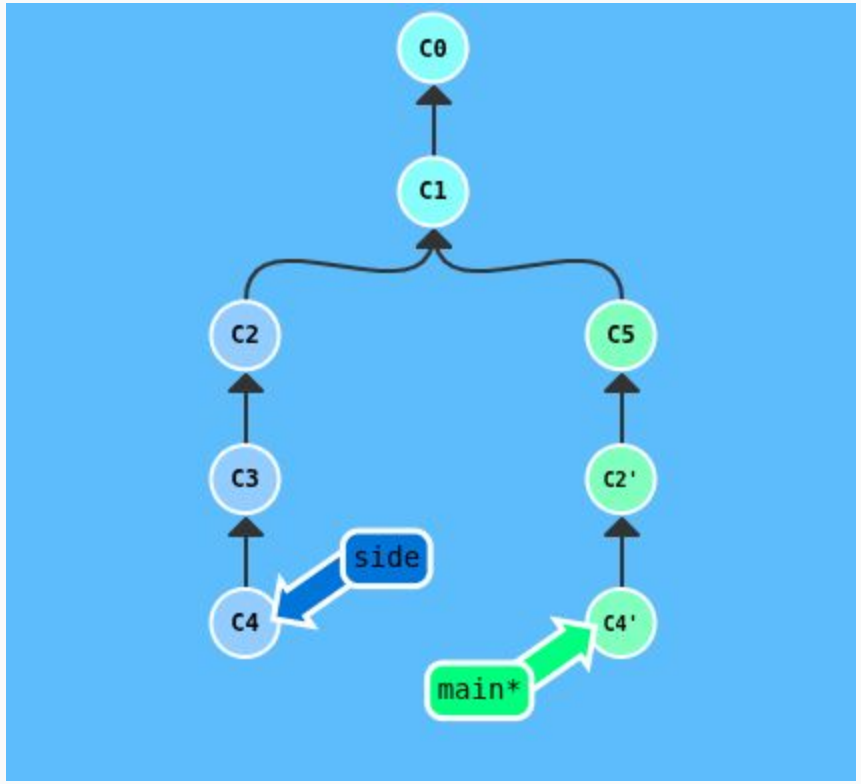
# git reset --hard <commit>

# Cherry Pick



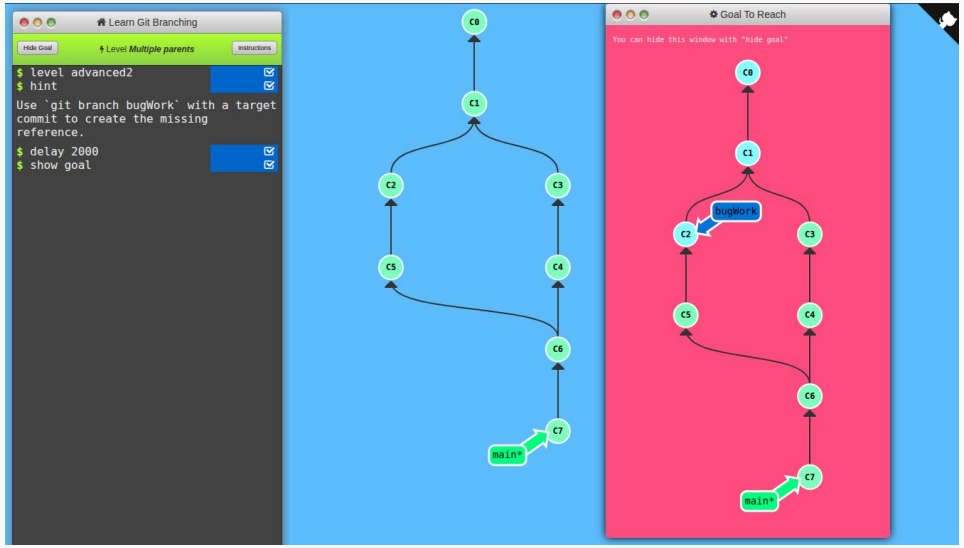
# Cherry Pick

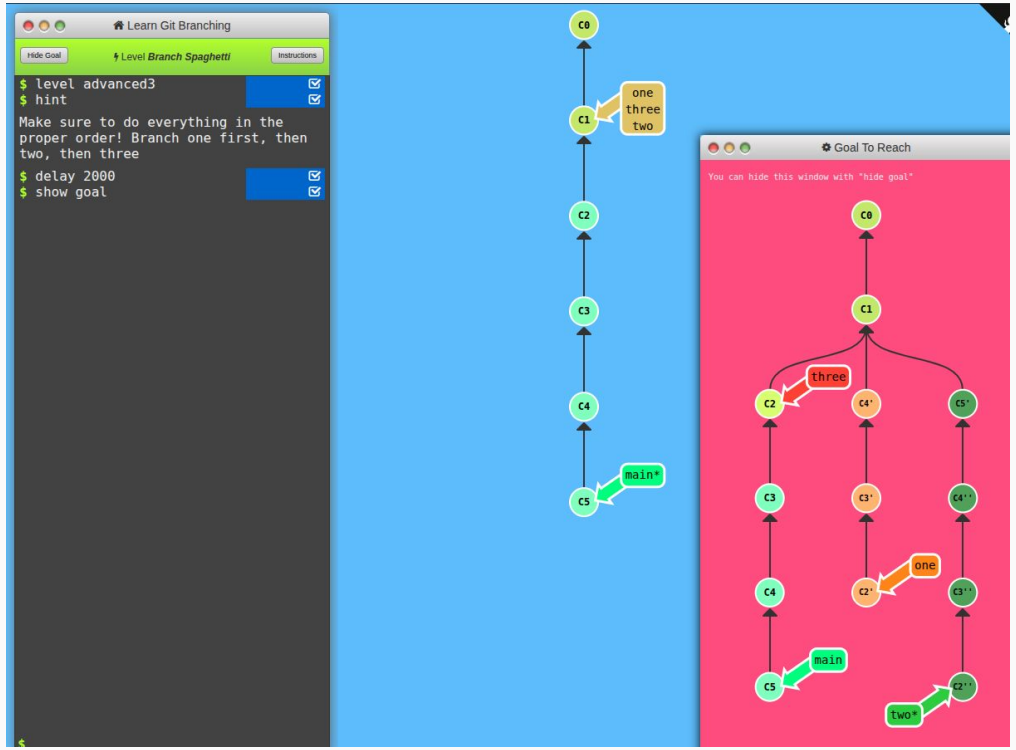




# https://learngitbranching.js.org/





Rabin Lamichhane

C

Assembly language

Full stack development

Microprocessor

Embedded system